

PlateletIQ — Complete System Architecture & Specification

End-to-End Implementation, Two-Track Architecture, Technical Choices & Data Pipeline

1. System Overview & Two-Track Architecture

PlateletIQ is engineered around a **Decoupled Two-Track Data Architecture** designed to separate machine learning demand forecasting from regional supply-chain visibility. This ensures strict data provenance and prevents unverified external data from corrupting clinical predictions.

Track	Data Source	Screens	Provenance Badge
Track 1: Forecasting & Decision Engine	German Hospital Dataset (Aachen 2008–2018, 4,018 days) + WHO India Index	Overview, Forecast, Actions, Inventory, Data Setup	MODEL / SIMULATED / DEMO
Track 2: Regional Network Scraper	eRaktKosh National Portal Scraper Database (756 hospitals across 32 Indian districts)	Network, Transfers & Recovery	EXTERNAL

2. Technology Stack & Key Technical Choices

Layer	Technologies	Architectural Justification
Frontend	React 19, TypeScript, Vite 8, Tailwind CSS 4, TanStack Query v5, Recharts 3, Framer Motion 11, react-globe.gl	TanStack Query eliminates Redux complexity by managing server state caching and auto-invalidation. Framer Motion provides Apple HIG micro-interactions.
Backend	Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2.0, Uvicorn	FastAPI delivers async-native performance, built-in OpenAPI documentation (/docs), and strict typing via Pydantic v2 schemas.
Machine Learning	scikit-learn LASSO Regression (plateletiq_model.joblib, 2.3 KB artifact)	LASSO achieved MASE 0.734 (vs Naive 0.993 and Schilling et al. 0.746). Interpretable linear coefficients permit clinical auditability over opaque black-box neural nets.
Database	SQLite (local dev plateletiq.db) / Supabase PostgreSQL 15 (prod via DATABASE_URL) + eraktkosh.db (35MB read-only scraper DB)	Dual DB setup allows zero-config offline execution via SQLite while supporting enterprise managed Postgres in production.

3. Core Backend Logic & Decision Engine

Deterministic Decision Engine (backend/services/decision_engine.py)

The decision engine calculates inventory net position relative to 7-day median demand (q50) and safety buffer target:

- **HOLD:** Usable Stock $\geq$ 7-day demand. Net Position $\geq$ 0. Action: No procurement needed.
- **PROCURE:** Usable Stock covers near-term lead time (2 days) but deficit occurs by day 4. Action: Procure $|\text{Net Position}| + \text{Safety Buffer}$ units.
- **COLLECT:** Usable Stock $<$ 2 days expected demand. Action: Trigger urgent mobile donation camp.
- **WHO Transfusion Guidelines Concordance:** Automatically flags non-concordant orders (prophylactic allowed if count $< 20 \times 10^9/L$, therapeutic if $< 50 \times 10^9/L$ with bleeding).
- **FIFO Unit Enforcement:** Enforces First-In, First-Out issuing based on unit expiry_at timestamp. Blocks out-of-order issuance unless an override reason is logged.

Regional Scraper & Freshness Services

- **eRaktKosh Service (eraktkosh.py):** Evaluates 756 hospitals to track SDP scarcity (4.7% national stock share) and zero-SDP districts (23 of 32 districts).
- **Freshness Categorization (freshness.py):** Maps data age to operational status: CURRENT (0-24h), AGING (24-72h), STALE (72-168h), VERY_STALE ($>168h$).

4. Ground Truth Parameters & Model Provenance

Metric	Value	Provenance Badge
PlateletIQ Forecast MASE	0.734	MODEL
Naive Baseline MASE	0.993	MODEL
Schilling Benchmark MASE	0.746	MODEL
Optimized Wastage Rate	3.25% (vs 9.61% baseline)	SIMULATED
Optimized Shortage Rate	3.04% (vs 6.73% baseline)	SIMULATED
Annual Cost Savings	Rs 7,20,000 / hospital	SIMULATED
National SDP Share	4.7% (23/32 districts zero SDP)	EXTERNAL
Training Dataset	4,018 days (Aachen 2008-2018)	MODEL

5. Verification & Production Deployment

- **Zero Fallbacks Assurance:** All static fallback objects (`{ ... }`) have been removed. API failures explicitly render an `ErrorState` component with a `Retry` button.
- **Vite Build Status:** `npm run build` completed with 0 errors in 2.16s across 1,386 modules.
- **Python Verification:** `py_compile` passed for all 9 routers, services, and models with 0 syntax errors.
- **Render Cloud Deployment Support:** Ready for Render Web Service (FastAPI running Uvicorn at `/api/v1/health`) and Render Static Site (React Vite frontend).

